Comprehension Questions: Function A2

Direction A: Identifying High-Level Stages

A1. Which option best describes the overall sequence of stages in the function?

A. Sample-count check → type inference → sparse conversion → return.

B. Reject special inputs → normalize dtype → process sparse/non-sparse branches → check common constraints → return.

C. Sparse conversion → string encoding → negative-value check → return.

D. dtype conversion → model training → array copying → return.

A2. When dtype='numeric' and the original input dtype is object, how does the code handle it?

A. Set dtype to None.

B. Set dtype to np.float64.

C. Convert the input directly to a string.

D. Raise an exception immediately.

A3. What role does the if sp.issparse(array) check play in the overall workflow?

A. It determines whether all subsequent checks are skipped.

B. It routes the input to different normalization paths for sparse and ordinary arrays.

C. It checks whether the array contains negative values.

D. It determines whether the original object is returned.

Direction B: Stage Transitions and Control Flow

B1. In the ordinary-array path, if the target dtype is integer but the converted array is floating-point, why does the code call _assert_all_finite first?

A. To prevent infinity or missing values from causing problems during integer conversion.

B. To determine whether the array is two-dimensional.

C. To force the creation of an array copy.

D. To check whether the array contains negative values.

B2. When ensure_2d=True and the converted input is a one-dimensional pandas Series, what happens?

A. It is automatically reshaped into a two-dimensional array.

B. The original Series is returned.

C. An exception is raised because a two-dimensional container is required.

D. It is converted into a sparse matrix.

B3. Which checks occur after the sparse and ordinary-array paths converge again?

A. URL encoding, port assembly, and path completion.

B. Minimum sample count, minimum feature count, non-negativity, and writability checks.

C. dtype inference only.

D. String-type checking only.

Direction C: Local Evidence and Module Boundaries

C1. Which location is the most suitable boundary between the dtype-preparation stage and the sparse/ordinary-array branching stage?

A. array_orig = array

B. if dtype is not None: dtype = np.dtype(dtype)

C. if sp.issparse(array):

D. return array

C2. Which code fragment best represents the responsibility of copying an array when necessary to avoid shared memory?

A. if copy and np.may_share_memory(array, array_orig): array = np.array(...)

B. if ensure_non_negative: check_non_negative(...)

C. if dtype_numeric: ...

D. _num_samples(array)

C3. Which option best describes the purpose of the force_writeable branch?

A. Unconditionally copy every input.

B. Copy the array when its underlying data is not writable so that the returned value is writable.

C. Force the array to use an integer dtype.

D. Force a sparse array to become an ordinary array.

Overall Question

T1. Which option best summarizes the function?

A. It only converts lists into NumPy arrays.

B. It normalizes types according to the input form, processes sparse or ordinary arrays through the appropriate path, and applies a series of common constraint checks.

C. It only checks whether the array contains missing values.

D. It only checks the number of samples and features.